JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.temporal

Enum ChronoField

java.lang.Object
    java.lang.Enum<ChronoField>
        java.time.temporal.ChronoField

All Implemented Interfaces:

Serializable, Comparable<ChronoField>, TemporalField

```
public enum ChronoField
extends Enum<ChronoField>
implements TemporalField
```

A standard set of fields.

This set of fields provide field-based access to manipulate a date, time or date-time. The standard set of fields can be extended by implementing TemporalField.

These fields are intended to be applicable in multiple calendar systems. For example, most non-ISO calendar systems define dates as a year, month and day, just with slightly different rules. The documentation of each field explains how it operates.

Implementation Requirements:

This is a final, immutable and thread-safe enum.

Since:

1.8

Enum Constant Summary

Enum Constants

Enum Constant	Description

`ALIGNED_DAY_OF_WEEK_IN_MONTH`	The aligned day-of-week within a month.
`ALIGNED_DAY_OF_WEEK_IN_YEAR`	The aligned day-of-week within a year.
`ALIGNED_WEEK_OF_MONTH`	The aligned week within a month.
`ALIGNED_WEEK_OF_YEAR`	The aligned week within a year.
`AMPM_OF_DAY`	The am-pm-of-day.
`CLOCK_HOUR_OF_AMPM`	The clock-hour-of-am-pm.
`CLOCK_HOUR_OF_DAY`	The clock-hour-of-day.
`DAY_OF_MONTH`	The day-of-month.
`DAY_OF_WEEK`	The day-of-week, such as Tuesday.
`DAY_OF_YEAR`	The day-of-year.
`EPOCH_DAY`	The epoch-day, based on the Java epoch of 1970-01-01 (ISO).
`ERA`	The era.
`HOUR_OF_AMPM`	The hour-of-am-pm.
`HOUR_OF_DAY`	The hour-of-day.
`INSTANT_SECONDS`	The instant epoch-seconds.
`MICRO_OF_DAY`	The micro-of-day.
`MICRO_OF_SECOND`	The micro-of-second.
`MILLI_OF_DAY`	The milli-of-day.
`MILLI_OF_SECOND`	The milli-of-second.
`MINUTE_OF_DAY`	The minute-of-day.
`MINUTE_OF_HOUR`	The minute-of-hour.
`MONTH_OF_YEAR`	The month-of-year, such as March.
`NANO_OF_DAY`	The nano-of-day.
`NANO_OF_SECOND`	The nano-of-second.
`OFFSET_SECONDS`	The offset from UTC/Greenwich.

PROLEPTIC_MONTH	The proleptic-month based, counting months sequentially from year 0.
SECOND_OF_DAY	The second-of-day.
SECOND_OF_MINUTE	The second-of-minute.
YEAR	The proleptic year, such as 2012.
YEAR_OF_ERA	The year within the era.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
<R extends **Temporal**> R	**adjustInto**(R temporal, long newValue)	Returns a copy of the specified temporal object with the value of this field set.
int	**checkValidIntValue**(long value)	Checks that the specified value is valid and fits in an `int`.
long	**checkValidValue**(long value)	Checks that the specified value is valid for this field.
TemporalUnit	**getBaseUnit**()	Gets the unit that the field is measured in.
String	**getDisplayName**(**Locale** locale)	

		Gets the display name for the field in the requested locale.
long	**getFrom**(**TemporalAccessor** temporal)	Gets the value of this field from the specified temporal object.
TemporalUnit	**getRangeUnit**()	Gets the range that the field is bound by.
boolean	**isDateBased**()	Checks if this field represents a component of a date.
boolean	**isSupportedBy**(**TemporalAccessor** temporal)	Checks if this field is supported by the temporal object.
boolean	**isTimeBased**()	Checks if this field represents a component of a time.
ValueRange	**range**()	Gets the range of valid values for the field.
ValueRange	**rangeRefinedBy**(**TemporalAccessor** temporal)	Get the range of

		valid values for this field using the temporal object to refine the result.
String	toString()	Returns the name of this enum constant, as contained in the declaration.
static ChronoField	valueOf(String name)	Returns the enum constant of this type with the specified name.
static ChronoField[]	values()	Returns an array containing the constants of this enum type, in the order they are declared.

Methods inherited from class java.lang.Enum

clone, compareTo, equals, finalize, getDeclaringClass, hashCode, name, ordinal, valueOf

Methods inherited from class java.lang.Object

getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.time.temporal.TemporalField

`resolve`

Enum Constant Detail

NANO_OF_SECOND

```
public static final ChronoField NANO_OF_SECOND
```

The nano-of-second.

This counts the nanosecond within the second, from 0 to 999,999,999. This field has the same meaning for all calendar systems.

This field is used to represent the nano-of-second handling any fraction of the second. Implementations of `TemporalAccessor` should provide a value for this field if they can return a value for `SECOND_OF_MINUTE`, `SECOND_OF_DAY` or `INSTANT_SECONDS` filling unknown precision with zero.

When this field is used for setting a value, it should set as much precision as the object stores, using integer division to remove excess precision. For example, if the `TemporalAccessor` stores time to millisecond precision, then the nano-of-second must be divided by 1,000,000 before replacing the milli-of-second.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode. The field is resolved in combination with `MILLI_OF_SECOND` and `MICRO_OF_SECOND`.

NANO_OF_DAY

```
public static final ChronoField NANO_OF_DAY
```

The nano-of-day.

This counts the nanosecond within the day, from 0 to (24 * 60 * 60 * 1,000,000,000) - 1. This field has the same meaning for all calendar systems.

This field is used to represent the nano-of-day handling any fraction of the second. Implementations of `TemporalAccessor` should provide a value for this field if they can return a value for `SECOND_OF_DAY` filling unknown precision with zero.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode. The value is split

to form NANO_OF_SECOND, SECOND_OF_MINUTE, MINUTE_OF_HOUR and HOUR_OF_DAY fields.

MICRO_OF_SECOND

```
public static final ChronoField MICRO_OF_SECOND
```

The micro-of-second.

This counts the microsecond within the second, from 0 to 999,999. This field has the same meaning for all calendar systems.

This field is used to represent the micro-of-second handling any fraction of the second. Implementations of TemporalAccessor should provide a value for this field if they can return a value for SECOND_OF_MINUTE, SECOND_OF_DAY or INSTANT_SECONDS filling unknown precision with zero.

When this field is used for setting a value, it should behave in the same way as setting NANO_OF_SECOND with the value multiplied by 1,000.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode. The field is resolved in combination with MILLI_OF_SECOND to produce NANO_OF_SECOND.

MICRO_OF_DAY

```
public static final ChronoField MICRO_OF_DAY
```

The micro-of-day.

This counts the microsecond within the day, from 0 to (24 * 60 * 60 * 1,000,000) - 1. This field has the same meaning for all calendar systems.

This field is used to represent the micro-of-day handling any fraction of the second. Implementations of TemporalAccessor should provide a value for this field if they can return a value for SECOND_OF_DAY filling unknown precision with zero.

When this field is used for setting a value, it should behave in the same way as setting NANO_OF_DAY with the value multiplied by 1,000.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode. The value is split to form MICRO_OF_SECOND, SECOND_OF_MINUTE, MINUTE_OF_HOUR and HOUR_OF_DAY fields.

MILLI_OF_SECOND

```
public static final ChronoField MILLI_OF_SECOND
```

The milli-of-second.

This counts the millisecond within the second, from 0 to 999. This field has the same meaning for all calendar systems.

This field is used to represent the milli-of-second handling any fraction of the second. Implementations of `TemporalAccessor` should provide a value for this field if they can return a value for `SECOND_OF_MINUTE`, `SECOND_OF_DAY` or `INSTANT_SECONDS` filling unknown precision with zero.

When this field is used for setting a value, it should behave in the same way as setting `NANO_OF_SECOND` with the value multiplied by 1,000,000.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode. The field is resolved in combination with `MICRO_OF_SECOND` to produce `NANO_OF_SECOND`.

MILLI_OF_DAY

```
public static final ChronoField MILLI_OF_DAY
```

The milli-of-day.

This counts the millisecond within the day, from 0 to (24 * 60 * 60 * 1,000) - 1. This field has the same meaning for all calendar systems.

This field is used to represent the milli-of-day handling any fraction of the second. Implementations of `TemporalAccessor` should provide a value for this field if they can return a value for `SECOND_OF_DAY` filling unknown precision with zero.

When this field is used for setting a value, it should behave in the same way as setting `NANO_OF_DAY` with the value multiplied by 1,000,000.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode. The value is split to form `MILLI_OF_SECOND`, `SECOND_OF_MINUTE`, `MINUTE_OF_HOUR` and `HOUR_OF_DAY` fields.

SECOND_OF_MINUTE

```
public static final ChronoField SECOND_OF_MINUTE
```

The second-of-minute.

This counts the second within the minute, from 0 to 59. This field has the same meaning for all calendar systems.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode.

SECOND_OF_DAY

```
public static final ChronoField SECOND_OF_DAY
```

The second-of-day.

This counts the second within the day, from 0 to (24 * 60 * 60) - 1. This field has the same meaning for all calendar systems.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode. The value is split to form `SECOND_OF_MINUTE`, `MINUTE_OF_HOUR` and `HOUR_OF_DAY` fields.

MINUTE_OF_HOUR

```
public static final ChronoField MINUTE_OF_HOUR
```

The minute-of-hour.

This counts the minute within the hour, from 0 to 59. This field has the same meaning for all calendar systems.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode.

MINUTE_OF_DAY

```
public static final ChronoField MINUTE_OF_DAY
```

The minute-of-day.

This counts the minute within the day, from 0 to (24 * 60) - 1. This field has the same meaning for all calendar systems.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode. The value is split to form `MINUTE_OF_HOUR` and `HOUR_OF_DAY` fields.

HOUR_OF_AMPM

```
public static final ChronoField HOUR_OF_AMPM
```

The hour-of-am-pm.

This counts the hour within the AM/PM, from 0 to 11. This is the hour that would be observed on a standard 12-hour digital clock. This field has the same meaning for all calendar systems.

When parsing this field it behaves equivalent to the following: The value is validated from 0 to 11 in strict and smart mode. In lenient mode the value is not validated. It is combined with `AMPM_OF_DAY` to form `HOUR_OF_DAY` by multiplying the {AMPM_OF_DAY} value by 12.

CLOCK_HOUR_OF_AMPM

```
public static final ChronoField CLOCK_HOUR_OF_AMPM
```

The clock-hour-of-am-pm.

This counts the hour within the AM/PM, from 1 to 12. This is the hour that would be observed on a standard 12-hour analog wall clock. This field has the same meaning for all calendar systems.

When parsing this field it behaves equivalent to the following: The value is validated from 1 to 12 in strict mode and from 0 to 12 in smart mode. In lenient mode the value is not validated. The field is converted to an `HOUR_OF_AMPM` with the same value, unless the value is 12, in which case it is converted to 0.

HOUR_OF_DAY

```
public static final ChronoField HOUR_OF_DAY
```

The hour-of-day.

This counts the hour within the day, from 0 to 23. This is the hour that would be observed on a standard 24-hour digital clock. This field has the same meaning for all calendar systems.

When parsing this field it behaves equivalent to the following: The value is validated in strict and smart mode but not in lenient mode. The field is combined with `MINUTE_OF_HOUR`, `SECOND_OF_MINUTE` and `NANO_OF_SECOND` to produce a `LocalTime`. In lenient mode, any excess days are added to the parsed date, or made available via `DateTimeFormatter.parsedExcessDays()`.

CLOCK_HOUR_OF_DAY

```
public static final ChronoField CLOCK_HOUR_OF_DAY
```

The clock-hour-of-day.

This counts the hour within the AM/PM, from 1 to 24. This is the hour that would be observed on a 24-hour analog wall clock. This field has the same meaning for all calendar systems.

When parsing this field it behaves equivalent to the following: The value is validated from 1 to 24 in strict mode and from 0 to 24 in smart mode. In lenient mode the value is not validated. The field is converted to an `HOUR_OF_DAY` with the same value, unless the value is 24, in which case it is converted to 0.

AMPM_OF_DAY

```
public static final ChronoField AMPM_OF_DAY
```

The am-pm-of-day.

This counts the AM/PM within the day, from 0 (AM) to 1 (PM). This field has the same meaning for all calendar systems.

When parsing this field it behaves equivalent to the following: The value is validated from 0 to 1 in strict and smart mode. In lenient mode the value is not validated. It is combined with `HOUR_OF_AMPM` to form `HOUR_OF_DAY` by multiplying the {AMPM_OF_DAY} value by 12.

DAY_OF_WEEK

```
public static final ChronoField DAY_OF_WEEK
```

The day-of-week, such as Tuesday.

This represents the standard concept of the day of the week. In the default ISO calendar system, this has values from Monday (1) to Sunday (7). The `DayOfWeek` class can be used to interpret the result.

Most non-ISO calendar systems also define a seven day week that aligns with ISO. Those calendar systems must also use the same numbering system, from Monday (1) to Sunday (7), which allows `DayOfWeek` to be used.

Calendar systems that do not have a standard seven day week should implement this field if they have a similar concept of named or numbered days within a period similar to a week. It is recommended that the numbering starts from 1.

ALIGNED_DAY_OF_WEEK_IN_MONTH

```
public static final ChronoField ALIGNED_DAY_OF_WEEK_IN_MONTH
```

The aligned day-of-week within a month.

This represents concept of the count of days within the period of a week where the weeks are aligned to the start of the month. This field is typically used with `ALIGNED_WEEK_OF_MONTH`.

For example, in a calendar systems with a seven day week, the first aligned-week-of-month starts on day-of-month 1, the second aligned-week starts on day-of-month 8, and so on. Within each of these aligned-weeks, the days are numbered from 1 to 7 and returned as the value of this field. As such, day-of-month 1 to 7 will have aligned-day-of-week values from 1 to 7. And day-of-month 8 to 14 will repeat this with aligned-day-of-week values from 1 to 7.

Calendar systems that do not have a seven day week should typically implement this field in the same way, but using the alternate week length.

ALIGNED_DAY_OF_WEEK_IN_YEAR

```
public static final ChronoField ALIGNED_DAY_OF_WEEK_IN_YEAR
```

The aligned day-of-week within a year.

This represents concept of the count of days within the period of a week where the weeks are aligned to the start of the year. This field is typically used with `ALIGNED_WEEK_OF_YEAR`.

For example, in a calendar systems with a seven day week, the first aligned-week-of-year starts on day-of-year 1, the second aligned-week starts on day-of-year 8, and so on. Within each of these aligned-weeks, the days are numbered from 1 to 7 and returned as the value of this field. As such, day-of-year 1 to 7 will have aligned-day-of-week values from 1 to 7. And day-of-year 8 to 14 will repeat this with aligned-day-of-week values from 1 to 7.

Calendar systems that do not have a seven day week should typically implement this field in the same way, but using the alternate week length.

DAY_OF_MONTH

```
public static final ChronoField DAY_OF_MONTH
```

The day-of-month.

This represents the concept of the day within the month. In the default ISO calendar system, this has values from 1 to 31 in most months. April, June, September, November have days from 1 to 30, while February has days from 1 to 28, or 29 in a leap year.

Non-ISO calendar systems should implement this field using the most recognized day-of-month values for users of the calendar system. Normally, this is a count of days from 1 to the length of the month.

DAY_OF_YEAR

```
public static final ChronoField DAY_OF_YEAR
```

The day-of-year.

This represents the concept of the day within the year. In the default ISO calendar system, this has values from 1 to 365 in standard years and 1 to 366 in leap years.

Non-ISO calendar systems should implement this field using the most recognized day-of-year values for users of the calendar system. Normally, this is a count of days from 1 to the length of the year.

Note that a non-ISO calendar system may have year numbering system that changes at a different point to the natural reset in the month numbering. An example of this is the Japanese calendar system where a change of era, which resets the year number to 1, can happen on any date. The era and year reset also cause the day-of-year to be reset to 1, but not the month-of-year or day-of-month.

EPOCH_DAY

```
public static final ChronoField EPOCH_DAY
```

The epoch-day, based on the Java epoch of 1970-01-01 (ISO).

This field is the sequential count of days where 1970-01-01 (ISO) is zero. Note that this uses the *local* time-line, ignoring offset and time-zone.

This field is strictly defined to have the same meaning in all calendar systems. This is necessary to ensure interoperation between calendars.

ALIGNED_WEEK_OF_MONTH

```
public static final ChronoField ALIGNED_WEEK_OF_MONTH
```

The aligned week within a month.

This represents concept of the count of weeks within the period of a month where the weeks are aligned to the start of the month. This field is typically used with `ALIGNED_DAY_OF_WEEK_IN_MONTH`.

For example, in a calendar systems with a seven day week, the first aligned-week-of-month starts on day-of-month 1, the second aligned-week starts on day-of-month 8, and so on. Thus, day-of-month values 1 to 7 are in aligned-week 1, while day-of-month values 8 to 14 are in aligned-week 2, and so on.

Calendar systems that do not have a seven day week should typically implement this field in the same way, but using the alternate week length.

ALIGNED_WEEK_OF_YEAR

```
public static final ChronoField ALIGNED_WEEK_OF_YEAR
```

The aligned week within a year.

This represents concept of the count of weeks within the period of a year where the weeks are aligned to the start of the year. This field is typically used with `ALIGNED_DAY_OF_WEEK_IN_YEAR`.

For example, in a calendar systems with a seven day week, the first aligned-week-of-year starts on day-of-year 1, the second aligned-week starts on day-of-year 8, and so on. Thus, day-of-year values 1 to 7 are in aligned-week 1, while day-of-year values 8 to 14 are in aligned-week 2, and so on.

Calendar systems that do not have a seven day week should typically implement this field in the same way, but using the alternate week length.

MONTH_OF_YEAR

```
public static final ChronoField MONTH_OF_YEAR
```

The month-of-year, such as March.

This represents the concept of the month within the year. In the default ISO calendar system, this has values from January (1) to December (12).

Non-ISO calendar systems should implement this field using the most recognized month-of-year values for users of the calendar system. Normally, this is a count of months starting from 1.

PROLEPTIC_MONTH

```
public static final ChronoField PROLEPTIC_MONTH
```

The proleptic-month based, counting months sequentially from year 0.

This field is the sequential count of months where the first month in proleptic-year zero has the value zero. Later months have increasingly larger values. Earlier months have increasingly small values. There are no gaps or breaks in the sequence of months. Note that this uses the *local* time-line, ignoring offset and time-zone.

In the default ISO calendar system, June 2012 would have the value `(2012 * 12 + 6 - 1)`. This field is primarily for internal use.

Non-ISO calendar systems must implement this field as per the definition above. It is just a simple zero-based count of elapsed months from the start of proleptic-year 0. All calendar systems with a full proleptic-year definition will have a year zero. If the calendar system has a minimum year that excludes year zero, then one must be extrapolated in order for this method to be defined.

YEAR_OF_ERA

```
public static final ChronoField YEAR_OF_ERA
```

The year within the era.

This represents the concept of the year within the era. This field is typically used with ERA.

The standard mental model for a date is based on three concepts - year, month and day. These map onto the YEAR, MONTH_OF_YEAR and DAY_OF_MONTH fields. Note that there is no reference to eras. The full model for a date requires four concepts - era, year, month and day. These map onto the ERA,

YEAR_OF_ERA, MONTH_OF_YEAR and DAY_OF_MONTH fields. Whether this field or YEAR is used depends on which mental model is being used. See ChronoLocalDate for more discussion on this topic.

In the default ISO calendar system, there are two eras defined, 'BCE' and 'CE'. The era 'CE' is the one currently in use and year-of-era runs from 1 to the maximum value. The era 'BCE' is the previous era, and the year-of-era runs backwards.

For example, subtracting a year each time yield the following:
- year-proleptic 2 = 'CE' year-of-era 2
- year-proleptic 1 = 'CE' year-of-era 1
- year-proleptic 0 = 'BCE' year-of-era 1
- year-proleptic -1 = 'BCE' year-of-era 2

Note that the ISO-8601 standard does not actually define eras. Note also that the ISO eras do not align with the well-known AD/BC eras due to the change between the Julian and Gregorian calendar systems.

Non-ISO calendar systems should implement this field using the most recognized year-of-era value for users of the calendar system. Since most calendar systems have only two eras, the year-of-era numbering approach will typically be the same as that used by the ISO calendar system. The year-of-era value should typically always be positive, however this is not required.

YEAR

```
public static final ChronoField YEAR
```

The proleptic year, such as 2012.

This represents the concept of the year, counting sequentially and using negative numbers. The proleptic year is not interpreted in terms of the era. See YEAR_OF_ERA for an example showing the mapping from proleptic year to year-of-era.

The standard mental model for a date is based on three concepts - year, month and day. These map onto the YEAR, MONTH_OF_YEAR and DAY_OF_MONTH fields. Note that there is no reference to eras. The full model for a date requires four concepts - era, year, month and day. These map onto the ERA, YEAR_OF_ERA, MONTH_OF_YEAR and DAY_OF_MONTH fields. Whether this field or YEAR_OF_ERA is used depends on which mental model is being used. See ChronoLocalDate for more discussion on this topic.

Non-ISO calendar systems should implement this field as follows. If the calendar system has only two eras, before and after a fixed date, then the proleptic-year value must be the same as the year-of-era value for the later era, and increasingly negative for the earlier era. If the calendar system has more than two eras, then the proleptic-year value may be defined with any

appropriate value, although defining it to be the same as ISO may be the best option.

ERA

```
public static final ChronoField ERA
```

The era.

This represents the concept of the era, which is the largest division of the time-line. This field is typically used with `YEAR_OF_ERA`.

In the default ISO calendar system, there are two eras defined, 'BCE' and 'CE'. The era 'CE' is the one currently in use and year-of-era runs from 1 to the maximum value. The era 'BCE' is the previous era, and the year-of-era runs backwards. See `YEAR_OF_ERA` for a full example.

Non-ISO calendar systems should implement this field to define eras. The value of the era that was active on 1970-01-01 (ISO) must be assigned the value 1. Earlier eras must have sequentially smaller values. Later eras must have sequentially larger values,

INSTANT_SECONDS

```
public static final ChronoField INSTANT_SECONDS
```

The instant epoch-seconds.

This represents the concept of the sequential count of seconds where 1970-01-01T00:00Z (ISO) is zero. This field may be used with `NANO_OF_SECOND` to represent the fraction of the second.

An `Instant` represents an instantaneous point on the time-line. On their own, an instant has insufficient information to allow a local date-time to be obtained. Only when paired with an offset or time-zone can the local date or time be calculated.

This field is strictly defined to have the same meaning in all calendar systems. This is necessary to ensure interoperation between calendars.

OFFSET_SECONDS

```
public static final ChronoField OFFSET_SECONDS
```

The offset from UTC/Greenwich.

This represents the concept of the offset in seconds of local time from UTC/Greenwich.

A `ZoneOffset` represents the period of time that local time differs from UTC/Greenwich. This is usually a fixed number of hours and minutes. It is equivalent to the `total amount` of the offset in seconds. For example, during the winter Paris has an offset of `+01:00`, which is 3600 seconds.

This field is strictly defined to have the same meaning in all calendar systems. This is necessary to ensure interoperation between calendars.

Method Detail

values

```
public static ChronoField[] values()
```

Returns an array containing the constants of this enum type, in the order they are declared. This method may be used to iterate over the constants as follows:

```
for (ChronoField c : ChronoField.values())
    System.out.println(c);
```

Returns:

an array containing the constants of this enum type, in the order they are declared

valueOf

```
public static ChronoField valueOf(String name)
```

Returns the enum constant of this type with the specified name. The string must match *exactly* an identifier used to declare an enum constant in this type. (Extraneous whitespace characters are not permitted.)

Parameters:

`name - the name of the enum constant to be returned.`

Returns:

`the enum constant with the specified name`

Throws:

`IllegalArgumentException - if this enum type has no constant with the specified name`

`NullPointerException - if the argument is null`

getDisplayName

```
public String getDisplayName(Locale locale)
```

Description copied from interface: `TemporalField`

Gets the display name for the field in the requested locale.

If there is no display name for the locale then a suitable default must be returned.

The default implementation must check the locale is not null and return `toString()`.

Specified by:

`getDisplayName in interface TemporalField`

Parameters:

`locale - the locale to use, not null`

Returns:

`the display name for the locale or a suitable default, not null`

getBaseUnit

```
public TemporalUnit getBaseUnit()
```

Description copied from interface: `TemporalField`

Gets the unit that the field is measured in.

The unit of the field is the period that varies within the range. For example, in the field 'MonthOfYear', the unit is 'Months'. See also `TemporalField.getRangeUnit()`.

Specified by:

`getBaseUnit` in interface `TemporalField`

Returns:

the unit defining the base unit of the field, not null

getRangeUnit

```
public TemporalUnit getRangeUnit()
```

Description copied from interface: `TemporalField`

Gets the range that the field is bound by.

The range of the field is the period that the field varies within. For example, in the field 'MonthOfYear', the range is 'Years'. See also `TemporalField.getBaseUnit()`.

The range is never null. For example, the 'Year' field is shorthand for 'YearOfForever'. It therefore has a unit of 'Years' and a range of 'Forever'.

Specified by:

`getRangeUnit` in interface `TemporalField`

Returns:

the unit defining the range of the field, not null

range

```
public ValueRange range()
```

Gets the range of valid values for the field.

All fields can be expressed as a `long` integer. This method returns an object that describes the valid range for that value.

This method returns the range of the field in the ISO-8601 calendar system. This range may be incorrect for other calendar systems. Use `Chronology.range(ChronoField)` to access the correct range for a different calendar system.

Note that the result only describes the minimum and maximum valid values and it is important not to read too much into them. For example, there could be values within the range that are invalid for the field.

Specified by:

`range` in interface `TemporalField`

Returns:

`the range of valid values for the field, not null`

isDateBased

```
public boolean isDateBased()
```

Checks if this field represents a component of a date.

Fields from day-of-week to era are date-based.

Specified by:

`isDateBased` in interface `TemporalField`

Returns:

`true if it is a component of a date`

isTimeBased

```
public boolean isTimeBased()
```

Checks if this field represents a component of a time.

Fields from nano-of-second to am-pm-of-day are time-based.

Specified by:

`isTimeBased` in interface `TemporalField`

Returns:

`true if it is a component of a time`

checkValidValue

```
public long checkValidValue(long value)
```

Checks that the specified value is valid for this field.

This validates that the value is within the outer range of valid values returned by `range()`.

This method checks against the range of the field in the ISO-8601 calendar system. This range may be incorrect for other calendar systems. Use `Chronology.range(ChronoField)` to access the correct range for a different calendar system.

Parameters:

value - the value to check

Returns:

the value that was passed in

checkValidIntValue

```
public int checkValidIntValue(long value)
```

Checks that the specified value is valid and fits in an `int`.

This validates that the value is within the outer range of valid values returned by `range()`. It also checks that all valid values are within the bounds of an `int`.

This method checks against the range of the field in the ISO-8601 calendar system. This range may be incorrect for other calendar systems. Use `Chronology.range(ChronoField)` to access the correct range for a different calendar system.

Parameters:

value - the value to check

Returns:

the value that was passed in

isSupportedBy

```
public boolean isSupportedBy(TemporalAccessor temporal)
```

Description copied from interface: `TemporalField`

Checks if this field is supported by the temporal object.

This determines whether the temporal accessor supports this field. If this returns false, then the temporal cannot be queried for this field.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `TemporalAccessor.isSupported(TemporalField)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisField.isSupportedBy(temporal);
temporal = temporal.isSupported(thisField);
```

It is recommended to use the second approach, `isSupported(TemporalField)`, as it is a lot clearer to read in code.

Implementations should determine whether they are supported using the fields available in `ChronoField`.

Specified by:

`isSupportedBy` in interface `TemporalField`

Parameters:

`temporal - the temporal object to query, not null`

Returns:

`true if the date-time can be queried for this field, false if not`

rangeRefinedBy

```
public ValueRange rangeRefinedBy(TemporalAccessor temporal)
```

Description copied from interface: `TemporalField`

Get the range of valid values for this field using the temporal object to refine the result.

This uses the temporal object to find the range of valid values for the field. This is similar to `TemporalField.range()`, however this method refines the result using the temporal. For example, if the field is `DAY_OF_MONTH` the `range` method is not accurate as there are four possible month lengths, 28, 29, 30 and 31 days. Using this method with a date allows the range to be accurate, returning just one of those four options.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `TemporalAccessor.range(TemporalField)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisField.rangeRefinedBy(temporal);
temporal = temporal.range(thisField);
```

It is recommended to use the second approach, `range(TemporalField)`, as it is a lot clearer to read in code.

Implementations should perform any queries or calculations using the fields available in `ChronoField`. If the field is not supported an `UnsupportedTemporalTypeException` must be thrown.

Specified by:

`rangeRefinedBy` in interface `TemporalField`

Parameters:

`temporal` - the temporal object used to refine the result, not null

Returns:

the range of valid values for this field, not null

getFrom

```
public long getFrom(TemporalAccessor temporal)
```

Description copied from interface: `TemporalField`

Gets the value of this field from the specified temporal object.

This queries the temporal object for the value of this field.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `TemporalAccessor.getLong(TemporalField)` (or `TemporalAccessor.get(TemporalField)`):

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisField.getFrom(temporal);
temporal = temporal.getLong(thisField);
```

It is recommended to use the second approach, `getLong(TemporalField)`, as it is a lot clearer to read in code.

Implementations should perform any queries or calculations using the fields available in `ChronoField`. If the field is not supported an `UnsupportedTemporalTypeException` must be thrown.

Specified by:

`getFrom` in interface `TemporalField`

Parameters:

`temporal - the temporal object to query, not null`

Returns:

`the value of this field, not null`

adjustInto

```
public <R extends Temporal> R adjustInto(R temporal,
                                         long newValue)
```

Description copied from interface: `TemporalField`

Returns a copy of the specified temporal object with the value of this field set.

This returns a new temporal object based on the specified one with the value for this field changed. For example, on a `LocalDate`, this could be used to set the year, month or day-of-month. The returned object has the same observable type as the specified object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then changing the month to February would be unclear. In cases like this, the implementation is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `Temporal.with(TemporalField, long)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisField.adjustInto(temporal);
temporal = temporal.with(thisField);
```

It is recommended to use the second approach, `with(TemporalField)`, as it is a lot clearer to read in code.

Implementations should perform any queries or calculations using the fields available in `ChronoField`. If the field is not supported an `UnsupportedTemporalTypeException` must be thrown.

Implementations must not alter the specified temporal object. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable implementations.

Specified by:

`adjustInto` in interface `TemporalField`

Type Parameters:

`R` - the type of the Temporal object

Parameters:

`temporal` - the temporal object to adjust, not null

`newValue` - the new value of the field

Returns:

the adjusted temporal object, not null

toString

`public String toString()`

Description copied from class: `Enum`

Returns the name of this enum constant, as contained in the declaration. This method may be overridden, though it typically isn't necessary or desirable. An enum type should override this method when a more "programmer-friendly" string form exists.

Specified by:

`toString` in interface `TemporalField`

Overrides:

`toString` in class `Enum<ChronoField>`

Returns:

the name of this enum constant

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.